Operationalizing the Four-Term Decomposition on AutoResearch:

Architecture Choice for Agentic Training Research
under Wall-Clock and Cost Budgets

Abstract

AutoResearch [2] is a concrete instance of a budgeted transductive task with a noisy scalar verifier: an agentic system modifies a single training script and is judged by held-out validation bits-per-byte after a fixed five-minute training run on a single H100. We apply the Beneventano–Poggio (BP) framework [1] to five architecture-choice questions: what "better" means, when one architecture beats another, why, by how much, and whether the answer can be predicted in advance. Three minimal extensions to BP are required and no more: (i) splitting certified time into wall-clock and cost axes, (ii) introducing a latent loss to absorb verifier noise, and (iii) folding context pressure into the per-step cost. The four-term decomposition $\Delta = \log(\kappa_0/\kappa) + \varphi + G - \varepsilon$ holds on both axes with the same three explanatory terms and a bifurcated cost term. We instantiate the framework on the stage-1 2×2 memory/parallelism ablation and specify a measurement protocol with falsifiable predictions.

1 Introduction

The meeting that motivated this note posed five architecture-choice questions for agentic systems on verifiable tasks:

Q1. What does it mean for architecture A to be better than architecture B?

Q2. When is A better than B?

Q3. Why is A better than B?

Q4. By how much is A better than B?

Q5. Can we predict in advance when A will be better than B?

Beneventano and Poggio (BP) [1] answer analogous questions in the setting of formal theorem proving in Lean 4, using a budgeted transductive task formalism, a packed latent-mode family, and a four-term decomposition

$$\Delta \;=\; \log\frac{\kappa_0}{\kappa} \;+\; \varphi \;+\; G \;-\; \varepsilon$$

that separates per-step cost, within-mode competence, information generation, and routing mismatch into independently measurable channels. Their setting assumes a deterministic verifier (the Lean kernel), a single fused certified time that collapses wall-clock and cost, and an implicit treatment of context budget.

AutoResearch is structurally close but differs in three ways that matter for the meeting's questions. The verifier, `val_bpb` after five minutes of training, is scalar and noisy. Architecture choice centers on parallel agents and shared workspace rather than model choice, which makes wall-clock genuinely distinct from cost. And the agent's context fills across attempts, which affects per-step latency in a way BP's constant κ cannot express.

We make three minimal extensions, then use BP's formalism as-is:

- Split certified time into T_{wall} and T_{cost} with parallel decompositions (Section 3).
- Replace the deterministic verifier with a latent loss $L(y) := \mathbb{E}_\omega[V(y,\omega)]$ and an incumbent re-evaluation protocol (Section 2).
- Fold context pressure into a c/K-dependent per-step cost, $\kappa(M, c/K)$, preserving the four-term structure (Section 4).

Sections 2–5 build the apparatus; Section 6 answers the five questions directly; Section 7 gives the measurement protocol; Section 8 situates the result.

2 AutoResearch as a budgeted transductive task

AutoResearch [2] instantiates a specific budgeted transductive task in the sense of BP Definition 1.

Definition 1 (AutoResearch task)**.** *The AutoResearch task is the tuple $\mathcal{T}_{AR} = (\mathcal{X}, \mathcal{Y}, V, B)$ where:*

- *$\mathcal{X} = \{x_0\}$ is the singleton instance given by the fixed* `nanochat` *dataset, tokenizer, and evaluation harness established by* `prepare.py`*;*
- *$\mathcal{Y}$ is the set of syntactically valid* `train.py` *files that compile and complete training within the platform budget;*
- *$V : \mathcal{Y} \times \Omega \to \mathbb{R}_{\geq 0}$ is the* `val_bpb` *score returned after a five-minute training run with training randomness $\omega \in \Omega$;*
- *$B = (b_t, b_c)$ specifies wall-clock and monetary budgets for the full agentic system.*

`val_bpb` is a continuous loss (lower is better), not a binary success indicator. The BP framework in its binary form does not directly apply; we use the continuous-reward extension of BP Appendix A, with one additional ingredient—the latent loss—to handle the noise.

Definition 2 (Latent loss)**.** *For candidate $y \in \mathcal{Y}$, the latent loss is*

$$L(y) \;:=\; \mathbb{E}_\omega[V(y,\omega)]\,.$$

A single evaluation $V(y,\omega)$ is an unbiased but noisy estimate of $L(y)$. Success at threshold $q^\star$ means producing y with $L(y) \leq q^\star$.

The latent loss is the clean object against which to define certified time. Any ranking of candidates based on single noisy evaluations is biased upward by selection; only repeated evaluation produces a consistent estimate. Section 7 specifies the incumbent re-evaluation protocol that operationalizes this.

Definition 3 (AutoResearch architecture)**.** *An AutoResearch architecture is a design tuple*

$$d \;=\; (M,\, \pi,\, \Omega,\, W,\, K,\, p)$$

in the BP sense (BP Definition 3), specialized to AutoResearch as follows: M is the base LLM (a single choice in our study); π is the orchestration/routing policy over edit strategies; Ω is the tool set (shell, file I/O, `uv run`*); W is the persistent external workspace (a directory of markdown files and optional cached training artifacts); K is the active context budget per LLM call; p is the number of parallel agents. We write A_d for the solver induced by d.*

The stage-1 comparison holds (M, Ω, K) fixed and varies (π, W, p). Four architectures instantiate the 2×2 baseline from the meeting transcript:

Definition 4 (The 2×2 baseline)**.**

$$\begin{aligned}
d_{00}: &\quad \textit{single agent, no external memory (workspace session-only)},\\
d_{10}: &\quad \textit{single agent, persistent external memory},\\
d_{01}: &\quad \textit{two parallel agents, no shared memory},\\
d_{11}: &\quad \textit{two parallel agents, shared external workspace.}
\end{aligned}$$

In BP coordinates: d_{00} *has* $(W = \varnothing, p = 1)$*;* d_{10} *has* $(W \neq \varnothing, p = 1)$*;* d_{01} *has* $(W = \varnothing, p = 2)$*;* d_{11} *has* $(W \neq \varnothing, p = 2)$*.*

This is the minimum design that isolates the memory main effect, the parallelism main effect, and their interaction.

3 Two certified times: wall-clock and cost

BP defines a single certified time $T^A(n, \delta)$. For AutoResearch, two things force a split. First, the five-minute training is a fixed wall-clock block per attempt, independent of the number of concurrent agents proposing in parallel. Second, running two agents in parallel roughly halves the wall-clock path for a given number of completed attempts but leaves the total token and GPU-hour cost approximately unchanged (in fact slightly higher due to coordination overhead). A single T^A cannot express both facts.

Definition 5 (Wall-clock and cost budgets)**.** *For a run of architecture* d *over a sequence of attempts, let* τ^d_{wall} *be the elapsed critical-path time from run start, including the longest dependency chain of LLM calls, file writes, and training evaluations. Let* τ^d_{cost} *be the cumulative monetary cost:*

$$\tau^d_{cost} \;=\; \underbrace{p_{LLM} \cdot \sum_{calls} tokens}_{agent\ tokens} \;+\; \underbrace{p_{GPU} \cdot GPU\text{-}hours}_{training\ cost},$$

where p_{LLM} *is the token price and* p_{GPU} *the hourly H100 rate.*

Definition 6 (Certified times on two axes)**.** *For target latent loss* $q^\star$ *and confidence level* $1 - \delta$*,*

$$\begin{aligned}
T^d_{wall}(q^\star, \delta) \;&=\; \inf\Big\{t : \mathbb{P}\Big[\exists s : \tau^d_{wall}(s) \le t,\ L(y_s) \le q^\star\Big] \ge 1 - \delta\Big\},\\
T^d_{cost}(q^\star, \delta) \;&=\; \inf\Big\{c : \mathbb{P}\Big[\exists s : \tau^d_{cost}(s) \le c,\ L(y_s) \le q^\star\Big] \ge 1 - \delta\Big\},
\end{aligned}$$

where y_s *is the* s*-th candidate produced by* A_d *and the probability is over training randomness* ω *and any internal randomness of the solver. The log-speedups are* $\Delta_{wall} = \log T^{d_0}_{wall} - \log T^d_{wall}$ *and* $\Delta_{cost} = \log T^{d_0}_{cost} - \log T^d_{cost}$*.*

Both are primary quantities. The point, spelled out more formally in Section 6, is that the two certified times may order architectures differently; when they do, the correct description is a Pareto trade-off, not a universal win.

4 Modes, context pressure, and the two per-step costs

We instantiate BP's packed latent-mode assumption (BP Assumption 6) with a fixed mode partition suited to AutoResearch.

Assumption 1 (AutoResearch modes)**.** *The latent mode $S \in \{1, \ldots, 6\}$ takes values in*

$$\{\textit{optimizer},\ \textit{LR schedule},\ \textit{architecture},\ \textit{regularization},\ \textit{data pipeline},\ \textit{other}\}$$

with prior π estimated empirically from successful edits. Each mode has a within-mode certified scale $t_0(M, s)$ giving the expected latent-loss improvement per five-minute attempt when the agent is routed to mode s. The baseline allocation $q_0 = \pi$ is prior-matched.

The mode partition is an analyst choice, as in BP. We discuss sensitivity in Section 8.

Context pressure enters through the per-step cost. As an agent accumulates attempts, its session context—the running diff of `train.py`, prior experimental results, critique of failed edits—fills toward its ceiling K. Both wall-clock per LLM call (via attention over longer context) and token cost (via input tokens) grow as a function of the context fill ratio $c/K \in [0, 1)$.

Definition 7 (Context-pressure-dependent per-step costs)**.** *Let c_t denote the active context fill at agent-step t and K the context ceiling. The per-step wall-clock and per-step cost functions are*

$$\begin{aligned}
\kappa_{wall}(M, c/K) &= \textit{wall-clock seconds per LLM call} + \textit{fixed evaluation time},\\
\kappa_{cost}(M, c/K) &= p_{LLM} \cdot \big(\textit{input tokens}(c) + \textit{output tokens}\big) + p_{GPU} \cdot (\textit{5 min}).
\end{aligned}$$

Both are non-decreasing in c/K. Averaged over a run, we write $\bar{\kappa}_{wall}(d)$ and $\bar{\kappa}_{cost}(d)$.

This folding is deliberate. An alternative—introducing context pressure as a fifth additive term in the decomposition—would break the clean algebraic structure. Treating c/K as a parameter of κ keeps the four-term form intact while making the pressure effect measurable: architectures that manage context well shift $\bar{\kappa}$ downward, and this shift appears in the cost term of the decomposition.

Remark 1 (Memory as κ-shifter)**.** In the 2×2 baseline, external memory in d_{10} and d_{11} does not primarily act through the information term G; it acts by lowering the effective c/K, because results offloaded to W need not live in the active context. The decomposition diagnoses this correctly only if κ is allowed to depend on context pressure.

5 The four-term decomposition for AutoResearch, on both axes

We now import BP Theorem 21 directly, apply it separately to each axis, and note that three of the four terms are shared.

Theorem 1 (Two-axis model-aware decomposition)**.** *Under Assumption 1 and the continuous-reward extension of BP Appendix A, with a prior-matched baseline architecture d_0 against a deployed architecture d,*

$$\Delta_{wall}(d) = \log \frac{\bar{\kappa}_{wall}(d_0)}{\bar{\kappa}_{wall}(d)} + \varphi(d_0, d) + G(d) - \varepsilon(d), \tag{1}$$

$$\Delta_{cost}(d) = \log \frac{\bar{\kappa}_{cost}(d_0)}{\bar{\kappa}_{cost}(d)} + \varphi(d_0, d) + G(d) - \varepsilon(d), \tag{2}$$

where φ, G, and ε are defined exactly as in BP Theorem 21 and take the same numerical values on both axes. Only the cost term differs.

Proof. Apply BP Lemma 18 and Theorem 21 to the log of each certified time separately. The log-linear structure $\log T_q(s) = \log \kappa + \log c_{\text{sim}} + \log t_0(M, s) - \log q(s)$ holds with κ replaced by κ_{wall} or κ_{cost}. The quantities $t_0(M, s)$, $G(M)$, and ε are structural features of the task and router and do not depend on which cost-conversion factor is applied to wall-clock seconds. The cross-entropy identity $\sum_s p(s) \log(1/q(s)) = H(p) + \mathrm{KL}(p \,\|\, q)$ separates the allocation into information and routing exactly as in BP, on both axes. □

The practical meaning is that a single stage-1 pilot estimates φ, G, and ε *once*; the two axes differ only in how each attempt is priced.

6 Answers to the five meeting questions

We can now address each question directly.

6.1 Q1. What does "better" mean?

Architecture d *dominates* d' at target $q^\star$ if it is at least as good on both axes:

$$T^d_{\text{wall}}(q^\star, \delta) \;\le\; T^{d'}_{\text{wall}}(q^\star, \delta) \quad \text{and} \quad T^d_{\text{cost}}(q^\star, \delta) \;\le\; T^{d'}_{\text{cost}}(q^\star, \delta),$$

with strict inequality on at least one. When neither dominates, we describe the pair as a *Pareto trade-off*. Equivalently, d dominates d' if and only if $\Delta_{\text{wall}}(d \mid d') \ge 0$ and $\Delta_{\text{cost}}(d \mid d') \ge 0$, with strict sign on at least one.

This is strictly stronger than a single-axis comparison and weaker than a scalar utility; it matches the meeting's insistence that the two axes be kept distinct. A paper reporting only wall-clock speedup hides cost-axis losses, and vice versa; reporting both makes Pareto disagreements visible and informative.

6.2 Q2. When is A better than B?

The 2×2 baseline gives three exact outcome-level contrasts per axis, following the pattern of BP Corollary 26 applied to the design variables (W, p) instead of to model choice. For a functional $F \in \{\Delta_{\text{wall}}, \Delta_{\text{cost}}\}$, define the memory main effect, parallelism main effect, and interaction:

$$\begin{aligned}
\mathrm{ME}^F_W &= \tfrac{1}{2}[(F_{00} - F_{10}) + (F_{01} - F_{11})]\,, \\
\mathrm{ME}^F_P &= \tfrac{1}{2}[(F_{00} - F_{01}) + (F_{10} - F_{11})]\,, \\
\mathrm{INT}^F &= (F_{00} - F_{10}) - (F_{01} - F_{11}).
\end{aligned}$$

Substituting Theorem 1 into each contrast gives the regime answer. For the memory main effect on the cost axis,

$$\mathrm{ME}^{\Delta_{\text{cost}}}_W \;=\; \underbrace{\tfrac{1}{2}\left[\log \tfrac{\bar\kappa_{\text{cost}\,00}}{\bar\kappa_{\text{cost}\,10}} + \log \tfrac{\bar\kappa_{\text{cost}\,01}}{\bar\kappa_{\text{cost}\,11}}\right]}_{\text{context-pressure relief}} + \Delta\varphi_W + \Delta G_W - \Delta\varepsilon_W,$$

and analogously on the wall-clock axis. The design statement is: *memory helps on axis F whenever context-pressure relief plus any information gain outweighs the routing and competence penalties.* The formula tells the builder which term to measure to determine the sign.

6.3 Q3. Why is A better than B?

The decomposition answers this by construction. Each of the four terms is measurable independently and maps to a distinct design lever:

- $\log(\bar{\kappa}_{\text{cost}0}/\bar{\kappa}_{\text{cost}})$: agent token price times tokens-per-call plus GPU-hours. The channel through which cheaper models and better context management win.
- $\log(\bar{\kappa}_{\text{wall}0}/\bar{\kappa}_{\text{wall}})$: latency per LLM call plus fixed training time. The channel through which parallelism and faster critical paths win.
- φ: within-mode competence, i.e. expected log-ratio of latent-loss improvements per mode. Fine-tuning and better reasoning capacity enter here.
- G: information generated by verifier feedback. Running more informative trials—diverse edits, early-stop signals, in-training curvature—raises G.
- ε: routing mismatch. Shared memory that broadcasts which modes are currently promising lowers ε by aligning the allocation with the true posterior.

  A diagnostic table for the 2×2:

Design variable	Primary affected term	Axis where it bites
External memory ($W \neq \varnothing$)	$\bar{\kappa}_{\text{cost}}, \bar{\kappa}_{\text{wall}}$ via c/K; also ε	both
Parallelism ($p = 2$)	$\bar{\kappa}_{\text{wall}}$ via critical path	wall-clock only
Shared workspace (W shared)	ε and diversity of attempts	both

6.4 Q4. By how much?

Theorem 1 gives the quantitative answer directly. Using illustrative values (a scaffold for magnitudes, to be replaced by pilot-measured values):

Design	$\log(\bar{\kappa}_{\text{cost}0}/\bar{\kappa}_{\text{cost}})$	$\log(\bar{\kappa}_{\text{wall}0}/\bar{\kappa}_{\text{wall}})$	φ	G	$-\varepsilon$	Δ_{cost} / Δ_{wall}
d_{10}	$+0.10$	$+0.10$	0	$+0.05$	0	$+0.15$ / $+0.15$
d_{01}	-0.10	$+0.60$	0	0	-0.05	-0.15 / $+0.55$
d_{11}	-0.05	$+0.65$	0	$+0.15$	$+0.20$	$+0.30$ / $+1.00$

(Illustrative only. Values in nats. Baseline is d_{00}.)

Reading the last row: d_{11} is $\exp(1.00) \approx 2.7\times$ faster than the single-agent baseline on wall-clock and roughly $\exp(0.30) \approx 1.35\times$ cheaper on cost. The wall-clock speedup is driven almost entirely by the parallelism term $+0.65$; the cost axis is kept positive only because the shared workspace reduces ε enough to offset the small cost-term loss from duplicated agent reasoning.

A paper reporting only "d_{11} is $2.7\times$ faster" loses exactly this sensitivity. If token prices drop by a factor of two, the cost term on d_{11} shifts by $\log 2 \approx 0.69$ nats, the cost-axis verdict becomes unambiguously positive, and the sensitivity of the conclusion to price assumptions becomes visible. The decomposition makes this traceable; a raw win rate does not.

6.5 Q5. Can we predict in advance when A will be better than B?

Specializing BP's crossover formula (BP Corollary 26) to the memory axis under AutoResearch coordinates: d_{10} dominates d_{00} on the cost axis whenever

$$\underbrace{\log \frac{\bar{\kappa}_{\text{cost}\,00}}{\bar{\kappa}_{\text{cost}\,10}}}_{\text{context-relief savings}} + \underbrace{\Delta\varepsilon_W}_{\text{routing gain}} > \underbrace{|\Delta G_W| + |\Delta\varphi_W|}_{\text{information/competence penalty, if any}} .$$

This is the condition needed to say, before running, that persistent external memory is expected to help. It translates the meeting's question "when does memory pay for itself?" into three quantities estimable from a pilot: the fractional token savings from offloading, the expected routing-mismatch reduction from re-exposing prior results at a later attempt, and any information/competence cost from the indirection.

For parallelism on the wall-clock axis, the prediction is even simpler. Under the packed-family assumption and with perfect work distribution, $p = 2$ roughly halves $\bar{\kappa}_{\text{wall}}$ (so the cost-term gain is about $\log 2 \approx 0.69$ nats) as long as the two agents do not redundantly propose the same edits—which is the routing-mismatch channel ε. The prediction is

$$\Delta_{\text{wall}}(d_{01} \mid d_{00}) \;\approx\; \log 2 - \Delta\varepsilon_{01|00}.$$

If the routing penalty from duplication exceeds $\log 2 \approx 0.69$ nats, two independent agents are worse than one on wall-clock, despite the apparent parallelism. This is measurable and falsifiable.

Putting these together yields an architecture-choice recipe without running the full 2×2: estimate $\bar{\kappa}_{\text{cost}}$ ratios from context-length measurements, estimate ε by comparing the empirical action distribution of two single-agent runs against a pooled posterior, and predict the sign of each contrast before committing to the full experiment. The predictions are not certain—the pilot protocol below is needed to validate them—but they are concrete enough to be wrong.

7 Measurement protocol for the 2×2 pilot

The decomposition is useful only if the four terms can be estimated from AutoResearch runs. We specify a protocol in the style of BP Section 9.

7.1 Mode labeling

Pool all successful edits (those that improved the latent `val_bpb` above the pre-edit incumbent) across development-set runs of d_{00} and d_{11}. Apply a domain-specific similarity metric: syntactic-tree distance on `train.py` diffs, augmented with keyword tags (`optim.`, `lr`, `arch`, `reg`, `data`). Hierarchical agglomerative clustering with bootstrap stability selection produces the empirical partition; retain the mode count that is stable across resamples, which for AutoResearch should land in the 4–8 range, compatible with Assumption 1. Fix labels before any model-dependent quantity is computed. Sensitivity to the partition is checked by rerunning the analysis with ± 1 merged or split clusters.

7.2 Event-level logging schema

Per LLM call: model id, start and end timestamps, input and output tokens, active context fill c_t, which agent emitted the call, and the full diff applied to `train.py` if any.

Per evaluation (`train.py` run): pre-run SHA of the file, five-minute start and end timestamps, final `val_bpb`, compile-time errors, OOM events, commit of `prepare.py` used.
Per attempt (one LLM-edit-evaluate cycle): attempt index, mode label (assigned post-hoc from the diff), cumulative agent-token cost, cumulative GPU-hours, whether this attempt updated the best-so-far.
Per run (one full architecture-level experiment): all of the above concatenated, plus the run-level τ^d_{wall} critical-path trace and the best-so-far curve as a function of τ^d_{wall} and τ^d_{cost}.

7.3 Incumbent re-evaluation for noise control

Because `val_bpb` is noisy, the best-so-far candidate cannot be identified from a single evaluation. The protocol: whenever an attempt s produces $V(y_s, \omega_s) < V_{\text{current-best}}$, re-run the training for candidate y_s two additional times to obtain $\hat{L}(y_s)$ as the mean of three evaluations; update the incumbent only if $\hat{L}(y_s) < \hat{L}(y_{\text{current-best}})$ by more than the pooled standard error. These re-evaluations are charged to a separate diagnostic budget distinct from the benchmark budget. This is the continuous-reward analogue of BP's δ-confident success criterion.

7.4 Budget equalization

Apples-to-apples comparison requires a single fairness condition, and wall-clock and cost must be fixed separately. We run the pilot under both:
Condition A (wall-clock-equalized): each architecture gets a fixed T-hour wall-clock budget, for T around 2 hours, matching Karpathy's baseline pace of roughly 12 attempts/hour. Single-agent designs complete about $12T$ attempts; two-agent designs complete about $24T$ attempts minus coordination overhead. Reports $T_{\text{wall}}(\cdot)$ and best latent loss at exhaustion.
Condition B (cost-equalized): each architecture gets a fixed total-cost budget in dollars, with the cost accounting of Definition 5. Reports $T_{\text{cost}}(\cdot)$ and best latent loss at exhaustion.

The two conditions will generically disagree on the ranking. This is the Pareto fact from Q1 in empirical form.

7.5 Estimators for the four terms

Following BP Table in spirit:

- $\bar{\kappa}_{\text{cost}}(d)$, $\bar{\kappa}_{\text{wall}}(d)$: direct averages from logs, stratified by context-fill bin.
- $\hat{\pi}$: empirical prior over modes from pooled successful attempts.
- $\varphi(d_0, d)$: $\sum_s \hat{\pi}(s) \log\big(\hat{t}_0(d_0, s)/\hat{t}_0(d, s)\big)$, where $\hat{t}_0$ is the median within-mode attempts-to-improvement.
- $\hat{G}(d) = H(\hat{\pi}) - \frac{1}{N}\sum_i H(\hat{\pi}_{D_i})$, with Miller–Madow correction for entropy bias.
- $\hat{\varepsilon}(d) = \frac{1}{N}\sum_i \mathrm{KL}(\hat{\pi}_{D_i} \,\|\, \hat{q}_{D_i})$ with Laplace smoothing.

Bootstrap confidence intervals accompany every quantity.

7.6 Testable predictions

Translating the meeting's hypotheses into predictions on the decomposition:

H1. Parallelism helps only on the wall-clock axis: $\Delta_{\text{wall}}(d_{01} \mid d_{00}) > 0$ while $\Delta_{\text{cost}}(d_{01} \mid d_{00}) \leq 0$. Specifically $\Delta_{\text{wall}}(d_{01} \mid d_{00}) \approx \log 2 - \Delta\varepsilon_{01|00}$.

H2. External memory helps on both axes through context-pressure relief: $\log(\bar{\kappa}_{00}/\bar{\kappa}_{10}) > 0$ on both $\bar{\kappa}_{\text{cost}}$ and $\bar{\kappa}_{\text{wall}}$, and the effect grows with attempt count.

H3. Shared memory lowers routing mismatch on multi-agent designs: $\varepsilon(d_{11}) < \varepsilon(d_{01})$, with magnitude comparable to $\log 2$ in the regime where d_{11} wall-clock-dominates d_{00}.

H4. The sign of Δ_{wall} on the parallelism contrast is sensitive to coordination efficiency: if $\varepsilon(d_{01}) - \varepsilon(d_{00}) > \log 2$, two independent agents lose on wall-clock.

H5. Context pressure is a dominant mechanism: stratifying $\bar{\kappa}$ by c/K bin shows monotone increase in per-attempt cost, and external memory's main effect is proportional to the average c/K difference between d_{00} and d_{10}.

7.7 Negative-result criterion

If, after the pilot, $\hat{\varphi}$, $\hat{G}$, and $\hat{\varepsilon}$ for the 2×2 cells do not exceed their bootstrap uncertainty, and the raw best-so-far curves in Conditions A and B are explainable by a single scalar efficiency metric with $R^2 > 0.9$, the decomposition has added no explanatory power in this domain and the paper should report this as a negative result.

7.8 Rough budget

At Karpathy's default pace of roughly 12 attempts/hour per agent, with a baseline of about \$0.02 per attempt in agent tokens plus \$0.20 per attempt in GPU (at an H100 spot rate of \$2.40/hour), a full 2×2 pilot under both conditions, run to 200 attempts per cell, costs approximately \$180 in compute and \$50 in agent tokens, for a total near \$230, plus the engineering cost of the logging harness and re-evaluation protocol. This is within the screening heuristic of BP Section 9.2.

8 Discussion

What the framework provides. Three things. First, a formal answer to "better" that respects the wall-clock/cost distinction the meeting insisted on. Second, an explanatory decomposition that traces each design variable to a measurable channel, so that "why did memory help" is not a vibe but a number. Third, predictive crossover formulas for each of the three design contrasts on each of the two axes, with only three structural quantities ($\bar{\kappa}$, ε, G) needing estimation from a pilot.

What it does not provide. The framework does not solve the joint design optimization. It does not tell the builder how to set prompting style, tool selection, or memory schema in advance—those remain engineering choices. It does not predict the result without a pilot; it structures the pilot's outputs so that a small study generalizes to nearby design points. And it does not cover cross-task transfer, since AutoResearch is a singleton task.

Sensitivity to the mode partition. The six-mode partition of Assumption 1 is a stipulation, not a discovery. If the empirical clustering of Section 7 produces a substantially different partition—say, four modes or nine—the numerical values of G and ε will change, but the qualitative decomposition remains valid as long as the partition is stable across bootstrap resamples. Predictions that flip sign under ± 1 cluster perturbation should be marked as unstable.

What the pilot will tell us. If the decomposition and crossover predictions hold, AutoResearch becomes the empirical foundation for extending BP beyond Lean: it will have been shown that mode-conditioned reasoning works in continuous-verifier settings with noisy evaluation. If the predictions fail—for instance, if context pressure turns out not to be the dominant mechanism, or

if mode labels are unstable across clustering choices—the failure will itself point to which of the three extensions was the wrong abstraction. Either outcome is informative.

The shortest honest summary of this note is: the meeting's five questions have clean answers in BP's language once wall-clock and cost are decoupled. The rest is instrumentation.

References

[1] P. Beneventano and T. Poggio. A mathematical framework for building agent systems: task-first design, equivalence, and model choice under deployment budgets. Under review, 2026.

[2] A. Karpathy. AutoResearch: AI agents running research on single-GPU `nanochat` training automatically. `https://github.com/karpathy/autoresearch`, 2026.

[3] A. Achille and S. Soatto. AI agents as universal task solvers: it's all about time. Preprint, 2026.